🧠 Derin Öğrenme Teori Master Rehberi (1 - 23 PDF Tam Kapsam)

> **Amaç:** Atıl Samancıoğlu'nun *100 Günde Derin Öğrenme* kursundaki 23 teorik PDF konusunu eksiksiz, mantıksal bir akışla, gündelik hayat benzetmeleriyle, matematiksel formülleriyle ve PyTorch mantığıyla tek bir usta başvuru kaynağında toplamak.

🗺️ İçindekiler ve 5 Büyük Mantıksal Blok

1. **Blok 1: Yapı Taşları ve Öğrenme Mekanizması (PDF 1 - 4)**
2. PDF 1: Perceptron (Tek Nöron)
3. PDF 2: Yapay Sinir Ağları (ANN Intro)
4. PDF 3: Geri Yayılım (Backpropagation)
5. PDF 4: Kayıp Gradyan Problemi (Vanishing Gradient)
6. **Blok 2: Aktivasyon Fonksiyonları (PDF 5 - 9)**
7. PDF 5: Sigmoid
8. PDF 6: TanH (Hyperbolic Tangent)
9. PDF 7: ReLU & GELU
10. PDF 8: Softmax
11. PDF 9: Aktivasyon Fonksiyonları Karşılaştırma Tablosu
12. **Blok 3: Hata Ölçümü - Loss & Cost Fonksiyonları (PDF 10 - 12)**
13. PDF 10: Loss vs. Cost Kavramı
14. PDF 11: Regresyon Hata Fonksiyonları (MSE, MAE, Huber)
15. PDF 12: Sınıflandırma Hata Fonksiyonları (Binary & Categorical Cross-Entropy)
16. **Blok 4: Öğrenme Algoritmaları - Optimizer'lar (PDF 13 - 17)**
17. PDF 13: Optimizer'lara Giriş ve Gradyan İnişi
18. PDF 14: SGD, Batch GD ve Momentum
19. PDF 15: Adagrad
20. PDF 16: RMSprop & Adadelta
21. PDF 17: Adam Optimizer
22. **Blok 5: Ağ Kararlılığı ve Ezberlemeyi (Overfitting) Önleme (PDF 18 - 23)**
23. PDF 18: Patlayan Gradyan Problemi (Exploding Gradient)

24. PDF 19: Ağırlık Başlatma (Weight Initialization - Xavier & He)
25. PDF 20: L1 & L2 Düzenlileştirme (Regularization / Weight Decay)
26. PDF 21: Dropout
27. PDF 22: Batch Normalization
28. PDF 23: Layer Normalization (Standart & Transformer)

---

1. Blok 1: Yapı Taşları ve Öğrenme Mekanizması (PDF 1 - 4)

PDF 1: Perceptron (Yapay Sinir Ağının En Temel Yapı Taşı)

- **Mantık:** Bir insanın karar verme mekanizmasının matematiksel modelidir. Girdileri ($x_1, x_2, \ldots$) alır, önem dereceleriyle (ağırlıklar: $w_1, w_2, \ldots$) çarpar, bir esneklik/eşik değeri (bias: b) ekler ve ham bir skor elde eder.
- **Matematiksel Formül:**

$$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b = \sum_{i=1}^{n} w_ix_i + b$$

$$\hat{y} = f(z) \quad (\text{Step / Basamak Fonksiyonu: } z > 0 \implies 1, \text{ aksi halde } 0)$$

- **Neden Bias (b) Şarttır?** Bias, karar sınırını kaydırmamızı sağlar. Bias 0 olursa karar çizgimiz orijinden $(0, 0)$ geçmek zorunda kalır.
- **Sınırlama:** Perceptron yalnızca **Lineer (doğrusal) ayrılabilir** problemleri çözebilir (örn: tek bir doğru çizerek kedi ve köpeği ayırmak). XOR gibi doğrusal olmayan problemleri **çözemez**.
- **PyTorch Karşılığı:** `torch.matmul(X, W) + b` veya `nn.Linear(in_features, 1)`.

PDF 2: Artificial Neural Networks (ANN Intro)

- **Mantık:** Tek bir perceptron XOR'u çözemiyorsa, nörondan bir "ordu" kurarız! Nöronları katmanlar halinde (Girdi, Gizli Katman, Çıktı) yan yana ve üst üste bağlarız.
- **Forward Propagation (İleri Yayılım):** Verinin soldan sağa doğru katman katman işlenerek nihai tahminin ($\hat{y}$) üretilmesidir.
- Ham Sinyal: $z^{(1)} = W^{(1)}x + b^{(1)}$
- Aktivasyon: $a^{(1)} = \sigma(z^{(1)})$
- Çıktı Katmanı: $z^{(2)} = W^{(2)}a^{(1)} + b^{(2)} \implies \hat{y} = \sigma(z^{(2)})$
- **Ağırlıklar Neden Sıfır Başlatılamaz?** Eğer tüm ağırlıkları 0 yaparsan, tüm nöronlar aynı sinyali üretir ve aynı şekilde güncellenir (*Symmetry Problem*). Bu yüzden küçük rastgele sayılarla başlatılır.

PDF 3: Backpropagation (Geri Yayılım)

- **Mantık:** Yapay zekanın "öğrenme" anıdır. Ağ bir tahmin yapar ($\hat{y}$), gerçek değerle (y) karşılaştırıp hatayı (Loss) hesaplar. Sonra sağdan sola doğru giderek: *"Tahmindeki hatada hangi ağırlık (W) ne kadar suçlu?"* sorusunun cevabını arar.
- **Matematiksel Temel (Zincir Kuralı / Chain Rule):** Bir W ağırlığının hataya etkisini (türevini) bulmak için türevleri zincirleme çarparız:

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial W}$$

- **Ağırlık Güncelleme (Gradient Descent):**

$$W_{\text{yeni}} = W_{\text{eski}} - \eta \cdot \frac{\partial L}{\partial W}$$

  Burada η (learning rate / öğrenme oranı), adım büyüklüğümüzdür.
- **PyTorch Karşılığı:** `loss.backward()` ve `optimizer.step()`.

PDF 4: Vanishing Gradient Problem (Kayıp Gradyan)

- **Problem:** Ağ derinleştikçe (katman sayısı arttıkça), Geri Yayılım yaparken türevler birbiriyle çarpılır. Eğer kullanılan aktivasyon fonksiyonunun (örn: Sigmoid) türevi 0.25'ten küçükse:

$$0.25 \times 0.25 \times 0.25 \times 0.25 \cdots \approx 0$$

- **Sonuç:** İlk katmanlardaki ağırlıkların türevi neredeyse 0 olur. Ağın giriş katmanları **öğrenmeyi tamamen durdurur**.
- **Çözüm:** Sigmoid/Tanh yerine türevi 0 olmayan **ReLU** kullanmak ve **Batch Normalization** eklemek.

---

2. Blok 2: Aktivasyon Fonksiyonları (PDF 5 - 9)

Aktivasyon fonksiyonları sinir ağına **Doğrusalsızlık (Non-linearity)** katar. Bunlar olmasaydı, 100 katmanlı bir devasa ağ bile matematiksel olarak tek bir doğrusal denklemden ibaret kalırdı.

PDF / Fonksiyon	Formül	Çıktı Aralığı	Avantajı	Dezavantajı / Kullanım Yeri
PDF 5: Sigmoid	$\sigma(z) = \frac{1}{1+e^{-z}}$	$(0, 1)$	Çıktıyı 0 ile 1 arasında olasılığa dönüştürür.	Satürasyon bölgesinde türevi $\to 0$ olur (Vanishing Gradient). Sıfır merkezli değildir. **İkili sınıflandırma (Binary) çıkışında kullanılır.**
PDF 6: Tanh	$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	$(-1, 1)$	Sıfır merkezlidir (Zero-centered), Sigmoid'den daha hızlı yakınsar.	Hala Vanishing Gradient problemi yaşar.
PDF 7: ReLU	$f(z) = \max(0, z)$	$[0, \infty)$	Hesaplaması aşırı hızlıdır. $z > 0$ için türevi 1'dir, **Vanishing Gradient'ı çözer!**	**Dying ReLU:** $z < 0$ durumunda türev 0 olduğu için nöron tamamen "ölebilir".
PDF 7: GELU	$f(z) = z \cdot \Phi(z)$	$(-0.17, \infty)$	Yumuşak türeve sahiptir, negatif değerlere küçük şans tanır.	**GPT, BERT, ViT (Transformer)** mimarilerinin standart aktivasyonudur.
PDF 8: Softmax	$P(y = i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$	$(0, 1)$ (Toplam=1)	Ham skorları (logits) birbirine bağımlı olasılık dağılımına dönüştürür.	**Çok sınıflı sınıflandırma (Multi-class) çıkış katmanında kullanılır.**

3. Blok 3: Hata Ölçümü - Loss & Cost Fonksiyonları (PDF 10 - 12)

- **PDF 10: Loss vs. Cost Farkı:**
- **Loss (Kayıp):** Tek bir veri örneği için yapılan tahmin hatası $L(\hat{y}, y)$.
- **Cost (Maliyet):** Tüm veri setindeki (veya mini-batch'teki) ortalama hata $J(W, b) = \frac{1}{N} \sum L_i$.

PDF 11: Regresyon Hata Fonksiyonları (Sayısal Tahmin)

1. **MSE (Mean Squared Error - Ortalama Kare Hata):**

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

Hataların karesini aldığı için büyük hataları aşırı cezalandırır. Outlier (aykırı değer) duyarlıdır.

2. **MAE (Mean Absolute Error - Ortalama Mutlak Hata):**

$$\text{MAE} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|$$

   Aykırı değerlere karşı daha dayanıklıdır (robust).
3. **Huber Loss:** Küçük hatalarda MSE, büyük hatalarda MAE gibi davranan hibrit yöntem.

PDF 12: Sınıflandırma Hata Fonksiyonları

1. **Binary Cross-Entropy (BCE - İkili Çapraz Entropi):**

$$\text{BCE} = -\frac{1}{N}\sum[y\log(\hat{y}) + (1-y)\log(1-\hat{y})]$$

   Kedi (1) / Köpek (0) gibi 2 sınıflı problemlerde kullanılır.
2. **Categorical Cross-Entropy (CCE):**

$$\text{CCE} = -\sum_{i} y_i \log(\hat{y}_i)$$

   Araba, Uçak, Gemi gibi K tane sınıf varsa Softmax sonrasında kullanılır.

---

4. Blok 4: Öğrenme Algoritmaları - Optimizer'lar (PDF 13 - 17)

Optimizer'ın amacı: Kayıp (Loss) dağının tepesinden, hatanın en düşük olduğu vadiye **en hızlı ve en kararlı** şekilde inmektir.

```
      SGD (En Basit / Zigzag çizer)
        ↓
    Momentum (Hız ve ivme katar)
        ↓
    Adagrad (Parametreye özel adaptif adım)
        ↓
RMSprop / Adadelta (Adagrad'ın adım küçülme sorununu çözer)
        ↓
     ADAM (Momentum + RMSprop Birleşimi - Kralların Optimizer'ı)
```

- **PDF 13 & 14: SGD & Momentum:**
- **SGD:** Her adımda yön değiştirip zigzag çizebilir.
- **Momentum:** Ağır bir topun vadiden aşağı yuvarlanırken ivme kazanması gibidir. Önceki adımların yönünü tutar ($v_t = \gamma v_{t-1} + \eta \nabla L$).
- **PDF 15: Adagrad:** Sık güncellenen parametrelerin adımını küçültür, nadir güncellenenlerin adımını büyütür. Ancak zamanla adım boyutu 0'a yakınsar ve öğrenme durur.

- **PDF 16: RMSprop:** Adagrad'ın adımının sıfırlanma sorununu geçmiş gradyanların hareketli ortalamasını (exponential moving average) alarak çözer.
- **PDF 17: Adam (Adaptive Moment Estimation):**
- **1. Moment (Ortalama):** Momentum'dan gelir.
- **2. Moment (Varyans):** RMSprop'tan gelir.
- *Sonuç:* Derin öğrenmedeki en popüler, en kararlı varsayılan optimizer'dır (`torch.optim.Adam`).

---

5. Blok 5: Ağ Kararlılığı ve Ezberlemeyi (Overfitting) Önleme (PDF 18 - 23)

PDF 18: Exploding Gradient (Patlayan Gradyan)

- **Problem:** Derin ağlarda veya RNN'lerde ağırlıklar 1'den büyükse, türevler çarpıla çarpıla sonsuza (∞) gider. Model çıktıları `NaN` verir.
- **Çözüm: Gradient Clipping** (Gradyanları belli bir eşik değerde kesmek, örn: $\text{max_norm}=1.0$).

PDF 19: Weight Initialization (Ağırlık Başlatma)

- **Xavier / Glorot Initialization:** Sigmoid ve Tanh kullanılan ağlarda varyansı koruyarak başlatır.
- **He / Kaiming Initialization:** ReLU ve türevleri kullanılan ağlarda varyansı korur. PyTorch'ta ReLU katmanları için standarttır.

PDF 20: L1 & L2 Regularization (Düzenlileştirme)

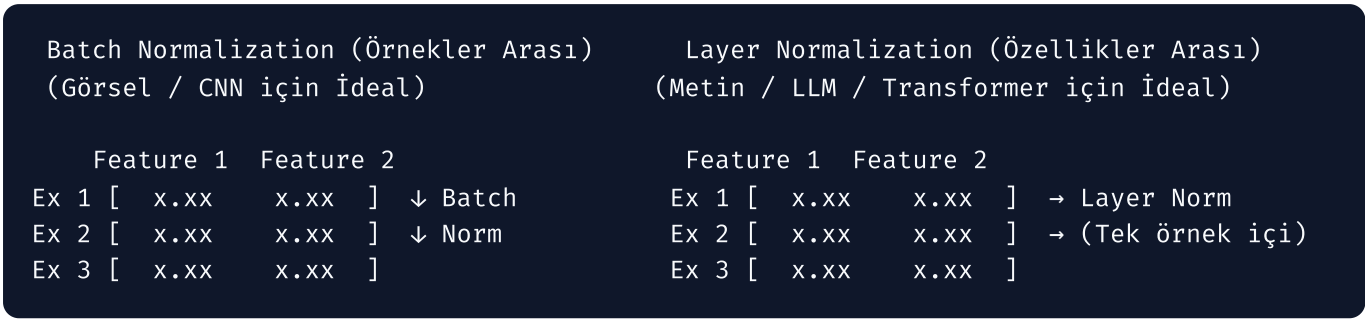
Ağın ezber yapmasını (Overfitting) önlemek için kayıp fonksiyonuna ceza terimi eklenir: * **L2 Regularization (Ridge / Weight Decay):** $\text{Ceza} = \lambda \sum W^2$. Ağırlıkları sıfıra yaklaştırır ama tam sıfır yapmaz. Ağırlıkları dengeli dağıtır. * **L1 Regularization (Lasso):** $\text{Ceza} = \lambda \sum |W|$. Gereksiz ağırlıkları **tam olarak 0** yapar (özellik seçimi sağlar).

PDF 21: Dropout

- **Mantık:** Eğitim sırasında her adımda nöronların $\%p$'sini (örn: $\%30$) **rastgele geçici olarak kapatırız (0 yaparız)**.
- **Neden İşe Yarar?** Nöronlar birbirine bel bağlayamaz (co-adaptation engellenir). Her nöron kendi başına bağımsız özellik öğrenmek zorunda kalır.
- **İnverted Dropout:** Eğitimde kalan nöronlar $\frac{1}{1-p}$ ile çarpılır ki test aşamasında hiçbir değişiklik yapmadan model doğrudan kullanılabilsin.
- **PyTorch Karşılığı:** `nn.Dropout(p=0.5)` . *(Önemli:* `model.train()` *iken aktiftir,* `model.eval()` *iken kendiliğinden kapanır!)*

PDF 22 & 23: Normalization (Batch Normalization vs. Layer Normalization)

Girdileri normalize etmek eğitimi hızlandırır. Peki katmanlar arası sinyalleri nasıl normalize edeceğiz?



1. **Batch Normalization (PDF 22):**
2. Her mini-batch içindeki verilerin aynı özelliğini (column) alır, ortalamasını 0, varyansını 1 yapar. Sonra öğrenilebilir γ ve β ile ölçekler.
3. **Avantajı:** Gradyan akışını düzenler, daha yüksek öğrenme oranı kullanmaya izin verir.
4. **Kullanım Yeri:** CNN ve Bilgisayarlı Görü (Vision) modellerinde standarttır (`nn.BatchNorm2d`).
5. **Layer Normalization (PDF 23):**
6. Batch boyutuna bağımlı DEĞİLDİR. Tek bir veri örneğinin tüm özelliklerini (row) kendi içinde normalize eder.
7. **Kullanım Yeri:** RNN'ler, NLP, **Transformers (GPT, Llama, ViT)** mimarilerinde standarttır (`nn.LayerNorm`).

🏁 Son Söz ve Zihinsel Check-List

Artık derin öğrenmenin **23 teorik köşe taşının tümü** elinin altında.

Kodlamaya geçtiğinde: - `nn.Linear` yazdığında → **PDF 1 & 2 (Perceptron & ANN)** - `nn.ReLU` yazdığında → **PDF 7 (Aktivasyon & Vanishing Gradient Çözümü)** - `nn.CrossEntropyLoss` yazdığında → **PDF 12 (Sınıflandırma Hatası)** - `optim.Adam` yazdığında → **PDF 17 (Momentum + RMSprop)** - `nn.Dropout` & `nn.BatchNorm1d` yazdığında → **PDF 21 & 22 (Ezber Önleme & Kararlılık)**

Gözünde canlanacak! İstediğin zaman bu rehbere dönüp bakabilirsin. 🚀